Università degli Studi di Verona

CORSO DI LAUREA IN INFORMATICA

Fondamenti dell'Informatica

Federico Brutti
federico.brutti@studenti.univr.it

Inserire citazione inerente alla materia

Contents

1 Introduzione **3**
1.1 Storia dell'informatica 3
1.2 Contestualizzazione 4
1.3 Problema dell'Informatica 5
1.4 Concetti di base richiesti 6

2 Automi e linguaggi formali **7**
2.1 Linguaggi formali 7
2.2 Automi a stati finiti 7
2.2.1 Automi a stati finiti deterministici 8
2.2.2 Automi a stati finiti non deterministici 11
2.3 Linguaggi liberi da contesto, forme normali e automi a pila 11
2.4 Classificazione di Chomsky 11

3 Teoria della calcolabilità **12**
3.1 Meet and fuck kingdom, steve 12
3.2 Meet 'n HUH!? 12

Chapter 1

Introduzione

1.1 Storia dell'informatica

L'**informatica**, abbreviazione di informazione automatica è una scienza nata negli anni '60 dalla fusione di vari aspetti scientifici e ingegneristici. In particolare ha un'origine **matematica**, dove si è definito che cosa sia "calcolare", e una **tecnologica**, determinante gli strumenti utilizzati per il calcolo. Quest'ultimo, nel senso moderno, nasce nell'epoca dell'Illuminismo ed è dovuto, in ambito del Determinismo, alla necessità di approssimare logaritmi e funzioni trigonometriche mediante polinomi, ottenendo un risultato univoco a partire da date condizioni iniziali.

Formalizzate queste tecniche nel '700, si vide successivamente la necessità di **meccanizzare** il calcolo. Il primo strumento funzionante fu la **macchina analitica**, creata da Charles Babbage e Ada Lovelace nel 19° secolo; proponeva un concetto rivoluzionario: la possibilità di programmazione per poter ritornare risultati in base ai parametri forniti. Poteva essere usata per sviluppare qualsiasi funzione di complessità e generalità arbitrarie, ripetendo operazioni sugli stessi elementi.

Le origini matematiche dell'informatica ebbero invece luogo con la necessità di verificare se la matematica fosse formalizzabile come insieme finito non contraddittorio di assiomi. Ciò non è possibile, perché ci si scontra inevitabilmente con le contraddizioni e gli assurdi. L'effettivo limite di ciò che è calcolabile e non fu definito con il **teorema di incompletezza** di Gödel; la restrizione del range di studio fu fondamentale per avere un'idea più chiara del dominio operativo. Andando avanti con gli anni, Alan Turing ideò l'omonima macchina, la cui idea principale era la possibilità interpretare il **calcolo** come sequenza anche infinita di sostituzioni di simboli: un algoritmo. Questo era naturalmente un programma finito dove ogni passo dipende da altrettante informazioni finite. Separa inoltre il controllo dalla memoria, ponendo le basi per quello che sarebbero diventati i calcolatori moderni. Oltre a ciò, insieme a Church definì la **Tesi di Church-Turing**, la quale afferma che se un problema è intuitivamente calcolabile, allora esisterà una

macchina di Turing (MDT) o un dispositivo equivalente[1] in grado di risolverlo, ovvero calcolarlo. L'ambito di studio dell'informatica è interamente incluso nell'insieme delle funzioni calcolabili, ovvero i problemi, le cui soluzioni sono fornite dagli algoritmi.

1.2 Contestualizzazione

Studiare la teoria della ricorsione e della calcolabilità è utile per fornire consapevolezza in merito ai limiti logici assoluti; insegna in primo luogo a riconoscere problemi impossibili, fornisce un criterio per capire quando fermarsi e permette di partire da assunzioni più chiare durante lo sviluppo software.

Tutte queste competenze sono in ogni caso usate durante il **calcolo**, formalmente definito come la risoluzione di un problema tramite un algoritmo, indipendentemente da quanto tempo o spazio è richiesto per la computazione. Come già menzionato, tuttavia, non sarà sempre possibile trovare una soluzione per ogni problema proposto, anche se formalmente definibile; per questa ragione, in termini pratici, non esistono programmi che verifichino la correttezza assoluta o la sicurezza di un dato software. Ciò che più si avvicina allo scenario di controllo ideale è l'**analisi del software**, ovvero la verifica del comportamento del programma in base a ciò che ci si aspetta. In particolar modo bisogna garantire:

- **Affidabilità**: Il software non deve andare in crash improvvisamente.
- **Sicurezza**: Il software, interfacciandosi con utenti esterni, non deve essere violato o manipolato.
- **Protezione**: Il software, se in controllo di sistemi critici, non deve causare incidenti.

Idealmente, dato un qualunque software con una data proprietà di interesse, esisterebbe un programma di analisi atto al calcolo automatico e preciso del soddisfacimento della stessa in tempo finito: il debugger ideale. Tuttavia, non è realistico che esista, poiché non abbiamo modo di verificare a priori se un programma termini o meno; in altre parole non può esistere:

$$\text{Per ogni programma } p \in \mathbb{L},\ \text{halt}(p) = \text{true} \iff p \text{ termina}$$

Ciò vale anche per un eventuale programma antivirus ideale, che segnalerebbe correttamente ogni malware nel caso in cui i relativi programmi in questione eseguissero compiti malevoli, come rubare o cifrare i dati. Questo perché non è possibile creare un algoritmo universale per un'analisi infallibile; tenendo a mente la definizione appena discussa, supponiamo un malware M che esegue un secondo programma P su un dato input x: se $P(x)$

[1] Strumenti come lambda-calcolo, funzioni ricorsive o linguaggi di programmazione turing completi.

termina, un file viene rimosso, altrimenti non arriva mai alla cancellazione. Non potendo sapere se un programma termini a priori, non sarà altrettanto possibile segnalarlo come malware, mostrando il limite logico.

Ancora più difficile sarebbe dimostrare che un software sta agendo in modo malevolo quando il discriminante è una proprietà semantica non banale, come la cifratura dei dati, nel caso dei ransomware. Da ciò, il **Teorema di Rice**: non esiste un algoritmo tale che per ogni programma sia capace di decidere automaticamente una proprietà semantica non banale, come per esempio la terminazione, fornire un risultato specifico, o determinare se due computazioni siano equivalenti.

Ciò non significa che tutto sia precluso; bisogna accettare i limiti imposti dall'insieme dei problemi calcolabili per lavorare esclusivamente su questi ultimi. In questi termini, non sarà possibile avere analisi automatiche corrette e complete, come anche ottenere risposte sempre precise, data dalla possibilità del verificarsi di falsi negativi e positivi. In ogni caso, esistono più iter di lavoro per l'analisi del software:

- **Model checking**: Controllo esaustivo di un modello del programma da analizzare.
- **Analisi statica**: Controllo automatico ed esaustivo sul programma senza che venga eseguito.
- **Testing**: Verifica di un sottoinsieme delle esecuzioni del sistema.
- **Antivirus firma e sandbox**: Sistema difensivo probabilistico, vulnerabile a stealthing.

1.3 Problema dell'Informatica

Prima di definire formalmente il problema che l'Informatica si pone di risolvere, è necessario chiarire alcuni concetti preliminari: sappiamo in primo luogo che il dominio di lavoro è un sottoinsieme dell'insieme dei numeri naturali $\mathbb{N}$ contenente le funzioni calcolabili, le quali sono definite come un insieme di coppie appartenenti alle parti di $\mathbb{N}$, ovvero:

$$f \in \mathcal{P}(\mathbb{N} \times \mathbb{N})$$

Per conoscere quante sono le funzioni del suddetto dominio, quindi la **cardinalità** dell'insieme $\mathcal{P}(\mathbb{N} \times \mathbb{N})$, notiamo che il numero di elementi all'interno di $\mathbb{N} \times \mathbb{N}$ ed $\mathbb{N}$ è equivalente e come conseguenza diretta sono entrambi numerabili e ordinabili. In questi termini esiste quindi una biezione univoca tra i due insiemi, ovvero che a ogni input corrisponde un output. La cardinalità dell'insieme delle parti $\mathcal{P}(\mathbb{N} \times \mathbb{N})$ sarà a sua volta equivalente a quella delle funzioni $\mathbb{N} \times \mathbb{N}$, dimostrando che esistono tante funzione quanti sottoinsiemi di $\mathbb{N}$.

Si ritiene ora necessario capire quanti algoritmi risolutivi esistano, definiti in termini logici come sequenze di simboli su un alfabeto Σ, il quale deve essere finito e non vuoto; esisteranno quindi tanti programmi quante sono le stringhe componibili con il suddetto alfabeto, il cui insieme è indicato con Σ^*. Per esempio:

$$\Sigma = \{a, b\} \implies \Sigma^* = \{\epsilon, a, b, aa, ab, ba, bb, aaa, ...\}$$

Dove ϵ rappresenta la stringa vuota. Si tratta di un insieme ordinabile privo di un limite superiore e in quanto tale può essere messo in relazione con quello dei naturali; ne concludiamo che:

$$|\Sigma^*| = |\mathbb{N}|$$

Dunque la cardinalità dell'insieme degli algoritmi è equivalente a quella dell'insieme dei naturali. Inoltre, per il **teorema di Cantor**, si può dimostrare come l'insieme delle parti avrà sempre cardinalità maggiore rispetto a quello intero; per esempio:

$$S = \{a, b\} \implies \mathcal{P}(S) = \{\{a\}, \{b\}, \emptyset, \{a, b\}\} \implies |S| = 2; |\mathcal{P}(S)| = 4$$

1.4 Concetti di base richiesti

Chapter 2

Automi e linguaggi formali

2.1 Linguaggi formali

Per **linguaggio formale** intendiamo un insieme di stringhe composte da simboli di un alfabeto Σ, definito come un insieme non vuoto di questi ultimi.

In particolare, un **simbolo** è un'entità primitiva astratta non definita formalmente, come lettere e cifre; una loro sequenza finita è detta **stringa**, la quale avrà una **lunghezza** $|w|$ che indica il numero di occorrenze di simboli che la compongono, per esempio:

$$abbba \implies |w| = 5$$

In questi termini è necessario notare come l'ambito di questa materia sarà tradurre il calcolo delle funzioni in relazioni di insieme. Esistono diversi tipi di linguaggi:

- **Linguaggio finito**: Composto da sole costanti, non richiede alcuna memoria e non ci sono input sconosciuti.
- **Linguaggio regolare**: Si traduce in una funzione lineare per la quale serve memoria, ma di quantità finita necessaria per calcolarla.
- **Linguaggio context free**: Non è conosciuta la quantità di memoria richiesta per calcolare la funzione, richiede una stack.
- **Linguaggio decidibile**: Funzione per la quale la stack non è sufficiente e si richiede ulteriore memoria a supporto.

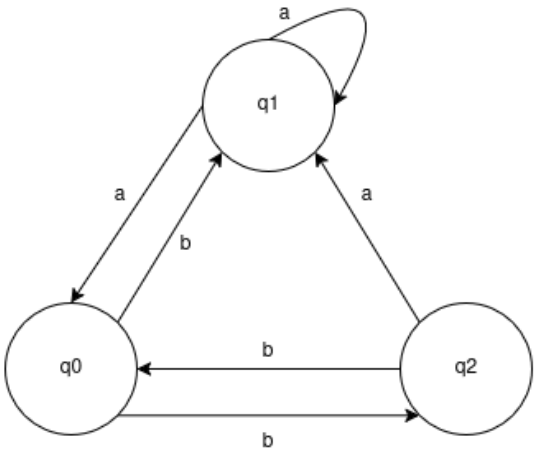
2.2 Automi a stati finiti

Un **Automa a stati finiti** è il modello di un sistema avente input e un eventuale output a valori discreti. Il sistema può trovarsi in uno stato all'interno di un loro insieme finito

e ciò gli consente di tenere traccia della storia precedente, quindi di avere una memoria finita.

Questi modelli sono rappresentabili mediante una testina che legge un nastro di lunghezza illimitata contenente i simboli. In base allo stato corrente q e del simbolo letto s_i, la testina vedrà se spostarsi in un altro stato o meno, e leggerà il simbolo successivo. Alla terminazione della lettura, in base allo stato raggiunto, l'automa fornirà un risultato di accettazione o rifiuto della stringa letta.

Il comportamento degli automi è definito univocamente con una tabella chiamata **matrice di transizione**, e sono rappresentati con il seguente grafo:



	a	b
q0	q1	q2
q1	q1	q0
q2	q1	q0

Figure 2.1: Esempio di automa a stati finiti

2.2.1 Automi a stati finiti deterministici

Gli Automi a stati finiti deterministici (DFA) sono una sottoinsieme di automi, definiti come una quintupla $M = (Q, \Sigma, \delta, q_0, F)$, i cui elementi sono:

- Q: Insieme finito degli stati.
- Σ: Alfabeto dei simboli che l'automa può riconoscere.
- $\delta : Q \times \Sigma \to Q$: Funzione di transizione degli stati, la quale, data una coppia (stato, simbolo), ritorna lo stato successivo.
- q_0: Stato iniziale dell'automa.
- $F \subseteq Q$: Insieme degli stati finali che descrive i risultati accettabili ottenibili dall'automa.

Dalla funzione di transizione δ si ottiene anche in modo univoco quella che descrive lo stato raggiunto dopo aver letto una sequenza di simboli $\hat{\delta} : Q \times \Sigma^* \to Q$ nel seguente modo:

$$\begin{cases} \hat{\delta}(q, \epsilon) = q \\ \hat{\delta}(q, \omega a) = \delta(\hat{\delta}(q, w)a) \end{cases}$$

Questa funzione è detta **chiusura transitiva di** δ; nel primo caso l'automa non legge nulla, mentre nella seconda espressione si indica che ha letto una sequenza di caratteri e un nuovo simbolo annotato con a.

Tuttavia, menzionando la possibilità di ritornare un risultato di non accettazione, ne consegue che esistano dei simboli (e di conseguenza, linguaggi), non riconoscibili dall'automa. Questa caratteristica è il discriminante che consente di definire i **linguaggi regolari**: tali se e solo se esiste un automa a stati finiti deterministico M che lo riconosce. Vengono definiti come:

$$L_m = \{\sigma \in \Sigma^* | \hat{\delta}(q_0, \sigma) \in F\}$$

L'insieme dei linguaggi caratterizzati dall'essere riconosciuti da un DFA è detto **insieme dei linguaggi regolari**, e per dimostrare che un linguaggio L gli appartiene è necessario fornire un DFA capace di riconoscerlo correttamente. Per esempio, supposto l'alfabeto $\Sigma = \{a, b\}$ e il linguaggio $\emptyset$, viene generato il seguente automa:

$$M_\emptyset = \langle \{q_0\}, \{a, b\}, \delta, q_0, \emptyset \rangle$$

Affermando che $\emptyset = L_m$, otteniamo:

$$L_m = \{\sigma \in Sigma^* | \hat{\delta}(q_0, \sigma) \in F\}$$

Semanticamente parlando, notiamo che è impossibile che l'automa si muova se non legge niente, infatti l'espressione $\hat{\delta}(q_0, \sigma) \in F$ è sempre falsa, definendo il corretto comportamento dell'automa quando non accade niente e provando che $\emptyset$ è un linguaggio regolare. Supponiamo ora un linguaggio di stringhe che contengono almeno due '1' sull'alfabeto $\Sigma = \{0, 1\}$, quindi:

$$L = \{x \in \Sigma^* | x \text{contiene due '1'}\}$$

Costruiamo un automa M a tre stati definito dalla seguente matrice di transizione:

	$i = 0$	$i = 1$
q_0	q_0	q_1
q_1	q_1	q_2
q_2	q_2	q_2

Table 2.1: Matrice di transizione

Questo deve riconoscere il linguaggio, il quale deve essere $L = L(m)$, che è equivalente a dire $L \subseteq L(m) \wedge L(m) \subseteq L$, ovvero tutto il linguaggio è riconosciuto da M e il linguaggio riconosciuto da M è contenuto in L. Dimostriamo le due tesi:

- $L \subseteq L(m) \iff \forall x \in \Sigma^*.x \in L \implies x \in L(m) \implies \hat{\delta}(q_0, x) \in F$

  Dunque per ogni stringa componibile con Σ^* è necessario che si riesca ad arrivare a uno stato finale.

- $L(m) \subseteq L \iff x \in L(m) \implies x \in L$

  Ovvero la stringa del linguaggio formale deve essere anche contenuta in quello che si vuole dimostrare tale.

Notare che per la seconda dimostrazione, la prima tesi vale se e solo se si riesce ad arrivare a uno stato finale con la stringa x, necessariamente contenuta in L. Poste queste basi per il linguaggio, possiamo procedere a dimostrare la regolarità di L per induzione sulla lunghezza di x, mostrando che:

$$\begin{cases} x \in L \implies \hat{\delta}(q_0, x) \in F \\ x \notin L \implies \hat{\delta}(q_0, x) \notin F \end{cases}$$

Passo base: $|x| = 0 \iff x = \epsilon$

Bisogna prendere la più piccola lunghezza che garantisca sia stringhe in L che stringhe ad esso non appartenenti. —x— = 2 è sufficiente. Procediamo a dimostrare la base per ogni lunghezza fino a due:

- $|x| = 0$:

$$\hat{\delta}(q_0, x) = \hat{\delta}(q_0, \epsilon) = q_0 \notin F$$

- $|x| = 1$:

$$\begin{cases} x = 0 \notin L & \implies \hat{\delta}(q_0, 0) = \delta(q_0, 0) = q_0 \notin F \\ x = 1 \notin L & \implies \hat{\delta}(q_0, 1) = \delta(1_0, 1) = q_1 \notin F \end{cases}$$

- $|x| = 2$:

$$\begin{cases} x = 00 \notin L & \implies \hat{\delta}(q_0, x) = q_0 \notin F \\ x = 01 \notin L & \implies \hat{\delta}(q_0, x) = q_1 \notin F \\ x = 10 \notin L & \implies \hat{\delta}(q_0, x) = q_1 \notin F \\ x = 11 \in L & \implies \hat{\delta}(q_0, x) = \delta(\hat{\delta}(q_0, 1), 1) = \delta(q_1, 1) = q_2 \in F \end{cases}$$

Questa è la definizione formale della base, per cui si dice che ogni stringa di lunghezza uguale a 2 che non contiene due cifre '1' non è accettata all'interno del linguaggio.

Passo induttivo: Sempre in riferimento alla lunghezza della stringa, è necessario dimostrare che per $|z| = n + 1$ valga la seguente ipotesi induttiva: sia $n \in N$; $\forall x.|x| \leq n$:

$$\begin{cases} x \in L \implies \hat{\delta}(q_0, x) \in F \\ x \notin L \implies \hat{\delta}(q_0, x) \notin F \end{cases}$$

Con una lunghezza della stringa non ben definita, è possibile che si abbiano $x = x'0$ oppure $x = x'1$, dipendentemente da cosa l'automa legge. Definiamo i casi:

- $x' \in L \implies x \in L$ per definizione, e per ipotesi induttiva porta allo stato finale:

$$\hat{\delta}(q_0, x') = q_2$$

- Se $x' \notin L$ e $x = x'1$ l'automa ha letto almeno una cifra '1'. Bisogna leggere il seguente simbolo. Se x' contiene '1', allora $x \in L$, altrimenti $x \notin L$.

- Se $x' \notin L$ e $x = x'0$ l'automa non ha ancora letto una cifra '1'; in questi termini non è possibile raggiungere uno stato finale, quindi varrà $x \notin L$ a prescindere dalla lettura di $x'0$ o $x'1$. Dunque:

$$\hat{\delta}(q_0, x) = \delta(\hat{\delta}(q_0, x'), 0) \in \{q_0 q_1\} \not\subseteq F$$

2.2.2 Automi a stati finiti non deterministici

2.3 Linguaggi liberi da contesto, forme normali e automi a pila

2.4 Classificazione di Chomsky

Chapter 3

Teoria della calcolabilità

3.1 Meet and fuck kingdom, steve

3.2 Meet 'n HUH!?